

EPIC 03- REST API DESIGN

Task 01 — REST API Fundamentals

You need to implement the following:

- Resources
- HTTP methods
- Status codes
- Request/response structure
- CRUD APIs

Here is a **simple Spring Boot implementation** you can create in Eclipse.

1. Create the Project

Create a Spring Boot project with:

- **Project:** Maven
- **Language:** Java
- **Java:** 17
- **Dependencies:** Spring Web

Example project name:

rest-api-fundamentals

2. Create a Model — Product.java

Create package:

`com.example.demo.model`

Create

`Product.java`:

```
package com.example.demo.model;
```

```
public class Product {

    private Long id;
    private String name;
    private double price;

    public Product() {
    }

    public Product(Long id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }
}
```

```
}  
}
```

3. Create REST Controller

Create package:

```
com.example.demo.controller
```

Create ProductController.java:

```
package com.example.demo.controller;  
  
import com.example.demo.model.Product;  
import org.springframework.http.HttpStatus;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;  
  
import java.util.ArrayList;  
import java.util.List;  
  
@RestController  
@RequestMapping("/api/products")  
public class ProductController {  
  
    private final List<Product> products = new ArrayList<>();  
  
    public ProductController() {  
        products.add(new Product(1L, "Laptop", 55000));  
        products.add(new Product(2L, "Mobile", 25000));  
    }  
  
    // GET - Read all products  
    @GetMapping  
    public ResponseEntity<List<Product>> getAllProducts() {  
        return ResponseEntity.ok(products);  
    }  
}
```

```

// GET - Read product by ID
@GetMapping("/{id}")
public ResponseEntity<Product> getProductById(@PathVariable Long id) {

    return products.stream()
        .filter(product -> product.getId().equals(id))
        .findFirst()
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}

// POST - Create product
@PostMapping
public ResponseEntity<Product> createProduct(@RequestBody Product product) {

    long newId = products.size() + 1L;
    product.setId(newId);
    products.add(product);

    return ResponseEntity.status(HttpStatus.CREATED).body(product);
}

// PUT - Update product
@PutMapping("/{id}")
public ResponseEntity<Product> updateProduct(
    @PathVariable Long id,
    @RequestBody Product updatedProduct) {

    for (Product product : products) {

        if (product.getId().equals(id)) {

            product.setName(updatedProduct.getName());
            product.setPrice(updatedProduct.getPrice());

            return ResponseEntity.ok(product);
        }
    }
}

```

```

        return ResponseEntity.notFound().build();
    }

    // DELETE - Delete product
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {

        boolean removed = products.removeIf(
            product -> product.getId().equals(id)
        );

        if (removed) {
            return ResponseEntity.noContent().build();
        }

        return ResponseEntity.notFound().build();
    }
}

```

4. Resources

In this API, **Product** is the resource.

The resource URL is:

/api/products

For a particular product:

/api/products/1

For example:

GET /api/products

means to get all products.

5. HTTP Methods

Method	Purpose	Example
GET	Read data	/api/products
POST	Create data	/api/products
PUT	Update data	/api/products/1
DELETE	Delete data	/api/products/1

These implement the main **CRUD** operations:

Create → POST

Read → GET

Update → PUT

Delete → DELETE

6. Status Codes

Your API should return appropriate HTTP status codes:

200 OK

The request was successful.

201 CREATED

A new product was successfully created.

204 NO CONTENT

The product was successfully deleted.

404 NOT FOUND

Products don't exist.

7. Request / Response Structure

POST Request

URL:

POST <http://localhost:8080/api/products>

Body:

```
{  
  "name": "Keyboard",  
  "price": 1500  
}
```

Response:

```
{  
  "id": 3,  
  "name": "Keyboard",  
  "price": 1500.0  
}
```

GET Request

GET <http://localhost:8080/api/products>

Response:

```
[  
  {  
    "id": 1,  
    "name": "Laptop",  
    "price": 55000.0  
  },  
]
```

```
{  
  
  "id": 2,  
  
  "name": "Mobile",  
  
  "price": 25000.0  
  
}  
]
```

8. Test the API

Start your Spring Boot application.

Then test using **Postman**.

GET all

GET <http://localhost:8080/api/products>

GET one

GET <http://localhost:8080/api/products/1>

POST

POST <http://localhost:8080/api/products>

Body → **raw** → **JSON**:

```
{  
  "name": "Keyboard",  
  "price": 1500  
}
```

PUT

PUT <http://localhost:8080/api/products/1>

Body:

```
{  
  "name": "Gaming Laptop",  
  "price": 65000  
}
```

DELETE

DELETE <http://localhost:8080/api/products/1>

Task 01 completed

This implementation demonstrates REST resources, HTTP methods, status codes, request/response structure, and complete CRUD APIs.

